



Κατανεμημένα Συστήματα

Εργαστηριακή Άσκηση 1

ΣΤ3-Α – Πέμπτη 14:00 - 16:00

Ρόμπερτ Πολοβίνα - 23390338

Εξάμηνο 8ο

2025-2026

Interface (.x) file και rpcgen utility

Ξεκινάμε ορίζοντας στο **.x** αρχείο τις τρεις συναρτήσεις που θα χρειαστούμε για να υλοποιήσουμε τα τρία ζητούμενα της άσκησης:

- A. Υπολογισμός μέσης τιμής διανύσματος Y .
- B. Μέγιστη και ελάχιστη τιμή του Y .
- C. Γινόμενο $a \cdot Y$.

Συνεπώς οι συναρτήσεις μας είναι αντίστοιχα:

- A. float **MEAN**(int_vector)
- B. minmax_result **MINMAX**(int_vector)
- C. float_vector **MULTIPLY**(scalar_vector_input)

Για την υλοποίηση των συναρτήσεων (και των υπολογισμών) θα χρειαστούμε επίσης και ειδικά διαμορφωμένους τύπους δεδομένων τους οποίους ορίζουμε για να παραχθούν μέσω του **rpcgen utility**. Οι τύποι αυτοί είναι:

- struct **int_vector**: Struct που περιέχει πίνακα (διάνυσμα) ακεραίων μεταβλητού μήκους **data<>**.
- struct **float_vector**: Ίδιο με το παραπάνω αλλά για floating-point αριθμούς.
- struct **minmax_result**: Ένα απλό struct για να ομαδοποιήσουμε τα αποτελέσματα εύρεσης μεγίστου και ελαχίστου.
- struct **scalar_vector_input**: Αποθηκεύει την μεταβλητή **a** η οποία απαιτείται για το 3ο ερώτημα, και το διάνυσμα που εισάγει ο χρήστης χρησιμοποιώντας τον τύπο **int_vector** που περιγράψαμε παραπάνω.

Έτσι καταλήγουμε στο παρακάτω **ask1.x** αρχείο:

```
struct int_vector{
    int data<>;
};

struct minmax_result{
    int min;
    int max;
};

struct scalar_vector_input{
    float a;
    int_vector vec;
};

struct float_vector{
    float data<>;
};
```

```
};

program CALC_PROG{
    version CALC_VERS{
        float MEAN(int_vector)= 1;
        minmax_result MINMAX(int_vector)= 2;
        float_vector MULTIPLY(scalar_vector_input)= 3;
    }= 1;
}= 0x164E882;
```

Όπου 0x164E882, μοναδικό program number το οποίο ισούται με 23390338 (AM).

Προχωράμε στην αυτοματοποιημένη παραγωγή των απαραίτητων system modules και templates μέσω του rpcgen utility με χρήση της εντολής **rpcgen -a -C ask1.x**. Το rpcgen utility μας παρήγαγε τα εξής αρχεία:

- **ask1.h**: Περιέχει τους ορισμούς των data types και των συναρτήσεων που γράψαμε στο αρχείο ask1.x, με τον τρόπο που απαιτεί το XDR.
- **ask1_clnt.c, ask1_client.c ask1_xdr.c, ask1_svc.c**: RPC client/server stub και XDR συναρτήσεις. Δεν τα επεξεργαζόμαστε.
- **ask1_server.c**: Template αρχείο το οποίο περιέχει τους ορισμούς των τριών συναρτήσεων που θα κάνουν τους απαραίτητους υπολογισμούς για κάθε ερώτημα. Τα σώματα των συναρτήσεων είναι κενά και συνεπώς τα συμπληρώνουμε όπως φαίνεται στον παρακάτω κώδικα.

ask1_server.c:

```
/*
 * This is sample code generated by rpcgen.
 * These are only templates and you can use them
 * as a guideline for developing your own functions.
 */

#include "ask1.h"
#include "common.h"

//Ερώτημα 1: Μέση Τιμή
float *
mean_1_svc(int_vector *argp, struct svc_req *rqstp)
{
    static float result;
    unsigned int i, n;
    int sum= 0;

    result= 0.0;
    n= argp->data.data_len;

    if(n== 0) return &result;

    for(i= 0; i< n; i++){
```

```

        sum+= argp->data.data_val[i];
    }

    result= (float)sum/n;
    return &result;
}

//Ερώτημα 2: Μέγιστο-Ελάχιστο
minmax_result *
minmax_1_svc(int_vector *argp, struct svc_req *rqstp)
{
    static minmax_result result;
    unsigned int i, n;

    result.min= 0;
    result.max= 0;

    n= argp->data.data_len;
    if(n== 0) return &result;

    result.min= argp->data.data_val[0];
    result.max= argp->data.data_val[0];

    for(i= 1; i< n; i++){
        if(argp->data.data_val[i]< result.min)
            result.min= argp->data.data_val[i];

        if(argp->data.data_val[i]> result.max)
            result.max= argp->data.data_val[i];
    }

    return &result;
}

//Ερώτημα 3: α*Υ
float_vector *
multiply_1_svc(scalar_vector_input *argp, struct svc_req *rqstp)
{
    static float_vector result;
    static float values[MAX_ELEMENTS];
    unsigned int i, n;

    result.data.data_val= values;
    result.data.data_len= 0;

    n= argp->vec.data.data_len;

    if(n> MAX_ELEMENTS)
        return &result;

    for(i= 0; i< n; i++){
        values[i]= argp->a*argp->vec.data.data_val[i];
    }

    result.data.data_len= n;

```

```
    return &result;
}
```

Παρατηρούμε πως η λογική του κώδικα των συναρτήσεων είναι απλός υπολογισμός των ζητούμενων. Η διαφορά (και εκεί που απαιτείται προσοχή) είναι στο ότι αντί για απλές μεταβλητές χρησιμοποιούμε τις δομές δεδομένων που ορίσαμε στην αρχή, δηλαδή το struct (vector) **data** εντός του struct **int_vector** και του **float_vector** αντίστοιχα, στην διαχείριση τους μέσω του ορίσματος **argp** και στην ομαδοποίηση των αποτελεσμάτων με χρήση του struct **minmax_result**. Η προσπέλαση στο **data** γίνεται ως εξής:

- **data.data_val**: Τα περιεχόμενα του διανύσματος (αρχή πίνακα).
- **data.data_val[i]**: Αριθμός (περιεχόμενο) του διανύσματος στην θέση **i**.
- **data.data_len**: Το μήκος του διανύσματος.
- **vec.data.data_val**: Ίδια λειτουργία αλλά το **data** ανήκει στο **int_vector** **vec**.

ask1.h:

```
struct int_vector {
    struct {
        u_int data_len;
        int *data_val;
    } data;
};
typedef struct int_vector int_vector;

struct minmax_result {
    int min;
    int max;
};
typedef struct minmax_result minmax_result;

struct scalar_vector_input {
    float a;
    int_vector vec;
};
typedef struct scalar_vector_input scalar_vector_input;

struct float_vector {
    struct {
        u_int data_len;
        float *data_val;
    } data;
};
typedef struct float_vector float_vector;
```

Σημαντικό είναι επίσης να προσέξουμε και να δηλώσουμε τις μεταβλητές των αποτελεσμάτων (**result**) ως **static** εφόσον τις επιστρέφουμε ως δείκτες, αλλιώς θα

δείχνουν σε τοπικές μεταβλητές οι οποίες θα πάψουν να υπάρχουν με το τέλος της συνάρτησης.

Server και Client

Για να επικοινωνήσουν, ο client και ο server πρέπει να ανταλλάξουν αρκετά δεδομένα (μεταβλητές). Για την ορθότερη διαχείριση τους ομαδοποιούμε τα δεδομένα αυτά σε δύο structs, το **Request** το οποίο περιέχει τα δεδομένα που στέλνει ο client στο server, και το **Response** που περιέχει τα δεδομένα που επιστρέφει (απάντα) ο server στον client. Τοποθετούμε τα δύο αυτά structs σε ένα header file, **common.h**, μαζί με άλλους δύο κοινούς ορισμούς (μέγιστο μήκος και port):

```
#ifndef COMMON_H
#define COMMON_H

#define MAX_ELEMENTS 100
#define PORT 3338

typedef struct{
    int choice;
    int n;
    int y[MAX_ELEMENTS];
    float a;
} Request;

typedef struct{
    int choice;
    int n;
    float mean;
    int min;
    int max;
    float result[MAX_ELEMENTS];
    int status;
} Response;
#endif
```

Τα structs αυτά είναι **fixed-size** και καλύπτουν και τις 3 περιπτώσεις. Όμως δεν χρησιμοποιούνται όλες οι μεταβλητές σε κάθε περίπτωση. Συνεπώς για την αποφυγή τυχόν σφαλμάτων, πάντα θα αρχικοποιούμε τα structs αυτά με την τιμή 0 μέσω της συνάρτησης **memset**, και θα χρησιμοποιούμε μόνο τα πεδία των structs που χρειάζονται ανάλογα την λειτουργία που επιλέγει ο χρήστης. Τέλος, το πεδίο **Response.status** μας είναι χρήσιμο για τον εντοπισμό σφαλμάτων και λόγω της αρχικοποίησης θέτουμε πως 0= οκ.

client_sock.c:

Ο κώδικας του client δεν περιέχει κάτι το ασυνήθιστο και δεν συμπεριλαμβάνεται ολόκληρος εδώ λόγω μεγέθους. Αποτελεί ένα “τυπικό” πρόγραμμα client με sockets (αρχικοποίηση, connect, write, read), ζητάει και διαβάζει δεδομένα από τον χρήστη, ελέγχει την ορθότητα τους, τα στέλνει στον server (write), δέχεται την απάντηση από τον server και τυπώνει τα αποτελέσματα. Προσοχή χρειάζεται για την σωστή διαχείριση των μεταβλητών μέσα στα structs που έχουμε ορίσει. Θα εστιάσουμε μόνο σε ορισμένα σημεία:

Αρχικοποίηση του **struct sockaddr_in server_addr**. Ως διεύθυνση βάζουμε την IP του localhost.

```
memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family= AF_INET;
server_addr.sin_port= htons(PORT);
server_addr.sin_addr.s_addr= inet_addr("127.0.0.1");
```

Αρχικοποίηση των structs **Request** και **Response** όπως εξηγήσαμε στην προηγούμενη ενότητα.

```
memset(&req, 0, sizeof(req));
memset(&resp, 0, sizeof(resp));
```

Αποστολή και λήψη δεδομένων στο/από το server (write – read) και έλεγχος του πεδίου **Response.status** για σφάλματα.

```
if(write(sock, &req, sizeof(req))< 0){
    perror("Client write error");
    break;
}
if(read(sock, &resp, sizeof(resp))<= 0){
    perror("Client read error");
    break;
}

if(resp.status!= 0){
    printf("Server returned error.\n");
    continue;
}
```

server_sock.c:

Και η δομή του server αποτελεί ένα “τυπικό” socket server πρόγραμμα. Αρχικοποιούμε το socket και το sockaddr_in struct, κάνουμε bind στο socket, αναμένουμε clients με listen και κάνουμε accept μέσα σε ατέρμονα βρόγχο σε συνδυασμό με **fork()** για να μπορούμε να εξυπηρετούμε πολλαπλούς clients ταυτόχρονα.

```
//Socket init
server_sock= socket(AF_INET, SOCK_STREAM, 0);
if(server_sock< 0){
    perror("Server socket error");
    exit(-1);
}

memset(&server_addr, 0, sizeof(server_addr));
server_addr.sin_family= AF_INET;
server_addr.sin_addr.s_addr= INADDR_ANY;
server_addr.sin_port= htons(PORT);
```

```
while(1){
    client_len= sizeof(client_addr);
    client_sock= accept(server_sock, (struct sockaddr *)&client_addr,
&client_len);
    if(client_sock< 0){
        perror("Server accept error");
        continue;
    }

    //Forking to handle multiple clients
    pid= fork();
    if(pid< 0){
        perror("Server fork error");
        close(client_sock);
        continue;
    }

    if(pid== 0){
        close(server_sock);
        manage_client(client_sock);
    }
    else{
        close(client_sock);
        //Zombie procc prevention
        while(waitpid(-1, NULL, WNOHANG)> 0);
    }
}
```

Διαχειριζόμαστε τους clients μέσα στην **manage_client()** συνάρτηση. Ορίζουμε τον client για το RPC μέσω της **clnt_create**, διαβάζουμε μέσω της **read()** τα δεδομένα που περιέχονται στο **Request** struct που στέλνει ο client και αναλόγως την επιλογή του χρήστη καλούμε την αντίστοιχη RPC συνάρτηση. Μας επιστρέφονται τα αποτελέσματα των υπολογισμών και τα στέλνουμε πίσω στον client μέσω της **write()**. Καθ' όλη τη διάρκεια της συνάρτησης εκτελούμε και τους απαραίτητους ελέγχους ορθότητας των δεδομένων.

Ιδιαίτερη προσοχή χρειάζεται και εδώ για την σωστή διαχείριση των μεταβλητών μέσω των structs που έχουμε δημιουργήσει.

Compilation & Makefile

Για να τρέξουμε το πρόγραμμα πρέπει να μεταγλωττίσουμε τρία αρχεία κώδικα:

- τον **rpc server** (ask1_server.c)
- τον **server** (server_sock.c)
- τον **client** (client_sock.c)

Θα χρειαστούν τρεις εντολές μεταγλώττισης οι οποίες περιέχουν και αρκετά ορίσματα. Συνεπώς, δημιουργούμε το παρακάτω αρχείο **Makefile**. Έτσι το μόνο που χρειάζεται να κάνουμε για να μεταγλωττίσουμε το πρόγραμμα μας είναι να γράψουμε απλώς **make** στη γραμμή εντολών.

Makefile:

```
CC = gcc
CFLAGS = -Wall -I/usr/include/tirpc
LIBS = -ltirpc

all: rpc_server socket_server socket_client

ask1.h ask1_clnt.c ask1_svc.c ask1_xdr.c ask1_client.c ask1_server.c: ask1.x
    rpcgen ask1.x

rpc_server: ask1_server.c ask1_svc.c ask1_xdr.c ask1.h
    $(CC) ask1_server.c ask1_svc.c ask1_xdr.c -o rpc_server $(CFLAGS) $(LIBS)

socket_server: server_sock.c ask1_clnt.c ask1_xdr.c ask1.h common.h
    $(CC) server_sock.c ask1_clnt.c ask1_xdr.c -o socket_server $(CFLAGS) $(LIBS)

socket_client: client_sock.c common.h
    $(CC) client_sock.c -o socket_client
```

Ενδεικτικά Τρεξίματα

Ανοίγουμε στο terminal τρία παράθυρα, ένα για το **rpc server**, ένα για το **server**, και ένα για το **client**, και εκτελούμε τις παρακάτω δοκιμές:

1) Μέση τιμή:

- επιλογή = 1
- n = 5
- Y = [2, 4, 6, 8, 10]

- Αναμενόμενο αποτέλεσμα: mean = 6.00

```

○ robert@R0-01D:~/DS$ ./socket_client

Διαθέσιμες ενέργειες:
1. Μέση τιμή διανύσματος Y
2. Μέγιστη και ελάχιστη τιμή του Y
3. Γινόμενο α*Y
0. Εξοδος
Επιλογή: 1
Εισάγεται μέγεθος n του διανύσματος (1-100): 5
Εισάγετε 5 ακέραια στοιχεία του Y:
Y[0]= 2
Y[1]= 4
Y[2]= 6
Y[3]= 8
Y[4]= 10
Μέση τιμή Y= 6.00

```

2) Min/Max:

- επιλογή = 2
- n = 6
- Y = [7, -3, 15, 0, 9, -1]
- Αναμενόμενο αποτέλεσμα = -3, 15

```

Διαθέσιμες ενέργειες:
1. Μέση τιμή διανύσματος Y
2. Μέγιστη και ελάχιστη τιμή του Y
3. Γινόμενο α*Y
0. Εξοδος
Επιλογή: 2
Εισάγεται μέγεθος n του διανύσματος (1-100): 6
Εισάγετε 6 ακέραια στοιχεία του Y:
Y[0]= 7
Y[1]= -3
Y[2]= 15
Y[3]= 0
Y[4]= 9
Y[5]= -1
Min= -3, Max= 15

```

3) $\alpha * Y$:

- επιλογή = 3
- $n = 4$
- $Y = [1, 2, 3, 4]$
- $\alpha = 2.5$
- Αναμενόμενο αποτέλεσμα = 2.50, 5.00, 7.50, 10.00

Διαθέσιμες ενέργειες:

1. Μέση τιμή διανύσματος Y
2. Μέγιστη και ελάχιστη τιμή του Y
3. Γινόμενο $\alpha * Y$
0. Εξοδος

Επιλογή: 3

Εισάγεται μέγεθος n του διανύσματος (1-100): 4

Εισάγετε 4 ακέραια στοιχεία του Y :

$Y[0] = 1$

$Y[1] = 2$

$Y[2] = 3$

$Y[3] = 4$

Εισάγετε πραγματικό αριθμό α : 2.5

$\alpha * Y = 2.50, 5.00, 7.50, 10.00$

4) Με μηδενικά και αρνητικούς:

- επιλογή = 1
- $n = 5$
- $Y = [-2, 0, 2, 4, 6]$
- Αναμενόμενο αποτέλεσμα: mean = 2.00

Διαθέσιμες ενέργειες:

1. Μέση τιμή διανύσματος Y
2. Μέγιστη και ελάχιστη τιμή του Y
3. Γινόμενο $a*Y$
0. Εξοδος

Επιλογή: 1

Εισάγεται μέγεθος n του διανύσματος (1-100): 5

Εισάγετε 5 ακέραια στοιχεία του Y :

$Y[0] = -2$

$Y[1] = 0$

$Y[2] = 2$

$Y[3] = 4$

$Y[4] = 6$

Μέση τιμή $Y = 2.00$